ReadSpace — Product Requirements & Technical Requirements

Working title: ReadSpace. A VS Code–style reader for notes and documents (HTML, Markdown, PDF, DOCX, TXT).

Version 1.0 · Prepared for Mujju · October 2026

How I read your sketch

Before the formal docs, here is what the hand-drawn skeleton shows, so we agree on the target:

- **Left panel (Explorer / Sidebar):** top label "File open", then a library/directory entry, then a **folder** with **files** nested inside it. This is a VS Code–style file tree for whatever folder the user opens.
- **Top strip (Tab bar):** a row of tabs `[file] [file] [file] ...` — each open file is a tab. The small mark under the first tab means "active tab".
- **Main area (Content / Reader):** the large area labelled "Cont" (content) where the selected file is rendered.
- **Top-right control inside content:** a **Preview / Edit** toggle.
- **Right edge:** a scroll indicator, meaning the content scrolls independently of the sidebar and tabs.
- **Vertical lines between panels:** draggable dividers, i.e. a **resizable, customizable layout**.

In short: *open a folder → browse files in a sidebar → open many files as tabs → read them in a clean viewer → resize and customize the layout.*

PART 1 — PRODUCT REQUIREMENTS DOCUMENT (PRD)

1. Overview

1.1 Problem

Students and developers keep notes and study material in many formats (Markdown, PDFs, Word files, HTML pages, plain text) spread across folders. Reading them means jumping between different apps (a PDF viewer, Word, a browser, a text editor). There is no single, distraction-free, VS Code–like workspace that treats a **folder of mixed documents as one readable project**.

1.2 Vision

One workspace where you open a folder, see its structure in a sidebar, open any supported document in a tab, and read it comfortably — with a layout you can resize and customize to your own taste.

1.3 Goals

1. Open a local folder and show its file/folder tree in a sidebar.
2. Read **HTML, Markdown, PDF, DOCX and TXT** inside the app with good rendering.
3. Support **multiple open files as tabs** with a familiar VS Code feel.
4. Provide a **resizable and customizable layout** (panels, themes, font size, reading width).
5. Provide a **Preview / Edit toggle** for text-based formats.
6. Feel fast, clean and keyboard-friendly.

1.3 Non-goals (v1)

- Not a full IDE: no debugging, terminal, extensions or git integration.
- No real-time collaboration or cloud sync.
- No editing of PDF or DOCX content (read-only for those in v1).
- No mobile-first layout (desktop and large tablets first).

2. Target users and personas

Persona	Description	Main need
Student (primary)	Keeps lecture notes, PYQs, PDFs and Markdown notes per course folder	Read everything for a course in one place, quickly switch between files
Developer	Has README/docs folders, HTML exports, design docs	Browse docs like in VS Code without opening an editor
Researcher / reader	Collects PDFs, Word reports and notes	Distraction-free reading with search and bookmarks

3. User stories

- As a user, I can **open a folder** and see all its subfolders and supported files in a sidebar tree.
- As a user, I can **click a file** and it opens in a new tab (single click = preview tab, double click = pinned tab).
- As a user, I can **switch, reorder and close tabs**, and the app remembers what I had open.
- As a user, I can **read a PDF** with page navigation, zoom and text search.
- As a user, I can **read Markdown** with headings, tables, code blocks and images rendered nicely.
- As a user, I can **read a DOCX** as formatted text.
- As a user, I can **view an HTML file** safely rendered.
- As a user, I can **toggle Preview / Edit** on Markdown, HTML and TXT, edit and save back to disk.
- As a user, I can **drag panel dividers** to resize the sidebar and content area, and collapse the sidebar.
- As a user, I can **customize** theme, font, font size, line height and content width.
- As a user, I can **search files** by name and **search inside** the current document.
- As a user, I can reopen the app and **restore my last workspace** (folder, tabs, layout).

4. Functional requirements

Priority: **P0** = must have for MVP, **P1** = should have, **P2** = nice to have.

4.1 Workspace and file explorer

ID	Requirement	Priority
FR-1	"Open Folder" action loads a local directory as the workspace	P0
FR-2	Sidebar shows a collapsible tree of folders and files, sorted folders-first then alphabetical	P0
FR-3	File icons by type (md, pdf, docx, html, txt)	P0
FR-4	Unsupported files are shown dimmed or hidden (user setting)	P1
FR-5	Filter box to search files by name in the tree	P1
FR-6	Refresh tree to pick up on-disk changes	P1
FR-7	Recent folders list on the welcome screen	P1
FR-8	Open multiple folders in one workspace	P2
FR-9	Context menu: reveal, copy path, close tab, rename, new file	P2

4.2 Tabs

ID	Requirement	Priority
FR-10	Each opened file appears as a tab in the top bar with its name and a close button	P0
FR-11	Active tab is visually highlighted	P0
FR-12	Single click opens a *preview* (italic) tab that is replaced by the next preview; double click	P1

ID	Requirement	Priority
	pins it	
FR-13	Drag to reorder tabs	P1
FR-14	Tab overflow scrolls horizontally with a dropdown listing all open tabs	P1
FR-15	Unsaved-changes dot on edited tabs; confirm on close	P0
FR-16	Keyboard: Ctrl+W close, Ctrl+Tab next, Ctrl+Shift+T reopen closed tab	P1
FR-17	Split view: two documents side by side	P2

4.3 Viewers (the core of the product)

ID	Format	Requirement	Priority
FR-20	Markdown (.md, .markdown)	Render GitHub-flavoured Markdown: headings, lists, tables, task lists, code with syntax highlighting, images (relative paths resolved), links	P0
FR-21	Markdown	Auto-generated table of contents (outline) from headings, click to scroll	P1
FR-22	PDF (.pdf)	Page rendering, continuous scroll, zoom, fit-width, page jump, text selection	P0
FR-23	PDF	In-document text search, thumbnails sidebar, remember last page	P1
FR-24	Word (.docx)	Convert to clean HTML and render with	P0

ID	Format	Requirement	Priority
		headings, lists, tables, images, bold/italic	
FR-25	Legacy Word (.doc)	Best-effort support through server-side conversion; otherwise show a clear message	P2
FR-26	HTML (.html, .htm)	Render safely in a sandbox, with option to view source	P0
FR-27	Text (.txt, .log, .csv as text)	Monospace or reading-font view with line numbers and wrap toggle	P0
FR-28	All	Find in document (Ctrl+F)	P1
FR-29	Images (.png, .jpg, .svg)	Basic image viewer	P2

4.4 Preview / Edit toggle

ID	Requirement	Priority
FR-30	Toggle button at the top-right of the content area switches between **Preview** and **Edit**	P0
FR-31	Edit mode available for Markdown, HTML and TXT using a code editor with syntax highlighting	P0
FR-32	Save with Ctrl+S writes back to disk (where the browser/OS permits)	P0
FR-33	Split mode: editor on the left, live preview on the right	P1
FR-34	Auto-save option	P2
FR-35	PDF and DOCX show Preview only; the toggle is hidden or disabled	P0

4.5 Layout and customization

ID	Requirement	Priority
FR-40	Draggable divider between sidebar and content; min/max widths enforced	P0
FR-41	Sidebar can be collapsed/expanded (Ctrl+B) and remembers its width	P0
FR-42	Content scrolls independently of the sidebar and tab bar	P0
FR-43	Light, dark and sepia themes; follows system theme by default	P0
FR-44	Reader settings: font family, font size, line height, max content width	P1
FR-45	Zen / focus mode that hides sidebar and tabs	P1
FR-46	Optional right-side panel (outline, bookmarks, document info)	P1
FR-47	Layout presets (Reading, Writing, Split) and saved custom layouts	P2
FR-48	Sidebar position left or right	P2

4.6 Persistence and productivity

ID	Requirement	Priority
FR-50	Restore last workspace: folder, open tabs, active tab, scroll positions, layout	P0
FR-51	Command palette (Ctrl+Shift+P) and quick open (Ctrl+P)	P1
FR-52	Bookmarks and highlights per document	P2

ID	Requirement	Priority
FR-53	Global search across all text-based files in the workspace	P2
FR-54	Export current Markdown or DOCX preview as PDF / print	P2

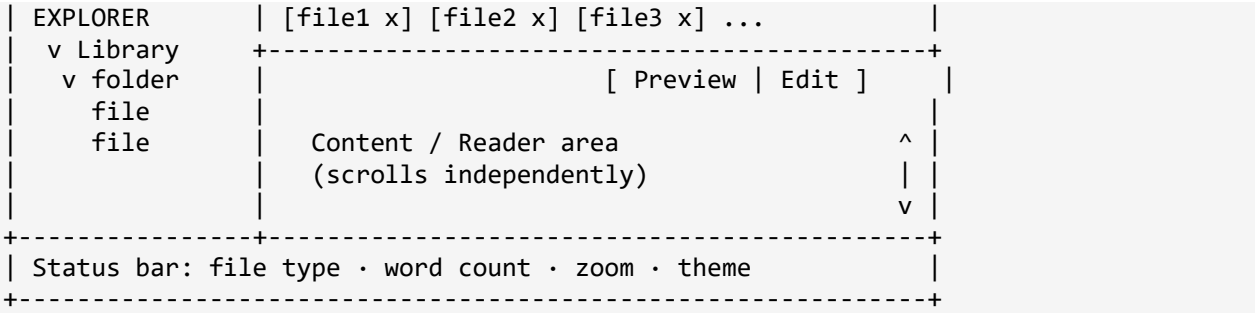
5. Non-functional requirements

Area	Requirement
Performance	Folder with 5,000 files shows the tree in under 1 second (lazy loading of subfolders). A 200-page PDF opens the first page in under 2 seconds. Markdown up to 1 MB renders in under 500 ms.
Responsiveness	UI stays at 60 fps while dragging dividers and scrolling; heavy parsing runs in Web Workers.
Privacy	All files stay on the user's device. No upload, no telemetry in v1.
Security	HTML and Markdown output sanitized; HTML files run in a sandboxed iframe with scripts disabled by default.
Reliability	No data loss: unsaved edits are cached locally and recoverable after a crash.
Accessibility	Full keyboard navigation, visible focus, ARIA roles for tree, tabs and panels, adequate contrast in all themes.
Compatibility	Latest Chrome and Edge for full features; Firefox and Safari with a reduced mode (see TRD). Desktop wrapper for Windows, macOS and Linux in a later phase.
Maintainability	TypeScript everywhere, modular renderer plugins, unit and e2e tests.

6. UX specification

6.1 Screen layout (from your sketch)

```
+-----------------------------------------------------------+
| Title bar / menu (File, View, Help)                       |
+---------------+-------------------------------------------+
```



- Vertical divider between Explorer and the content is draggable.
- An optional right panel (outline) can be toggled.
- The status bar is optional and added in P1.

6.2 Key interactions

- **Open folder:** welcome screen button or File → Open Folder.
- **Open file:** click in tree; opens as tab; active tab highlighted; sidebar highlights the same file.
- **Toggle Preview/Edit:** button at the top-right of the content; remembers the choice per file.
- **Resize:** hover divider shows resize cursor; double-click divider resets to default width.
- **Empty states:** friendly messages for no folder, no open tabs, unsupported file and conversion errors.

6.3 Design principles

- Calm, low-chrome interface that puts reading first.
- Familiar VS Code patterns so there is almost nothing to learn.
- Every preference has a sensible default; customization is optional.

7. Release plan

Phase	Scope	Outcome
MVP (v0.1)	Open folder, file tree, tabs, viewers for MD, TXT, HTML, PDF, DOCX, resizable sidebar, themes, Preview/Edit for text formats, save, restore session	Usable daily reader
v0.2	Outline panel, find in document, quick open, command palette, preview tabs, reader settings, zen mode	Comfortable power use
v0.3	Split view, layout presets, bookmarks, global search, export/print	Customization and productivity
v1.0	Desktop app (Tauri or Electron), legacy .doc conversion, auto-	Public release

Phase	Scope	Outcome
	update, polish and accessibility audit	

8. Success metrics

- Time from launch to reading a document under 10 seconds for returning users.
- At least 95 percent of test documents render without visible errors (test set of 50 mixed files).
- Zero crashes in a 30-minute session with 20 open tabs.
- User satisfaction of 4 out of 5 or better in a small test group of classmates.

9. Risks and mitigations

Risk	Impact	Mitigation
Browser file-system access differs by browser	Folder open may not work in Firefox/Safari	Use File System Access API with a fallback to directory upload; ship a desktop wrapper later
DOCX formatting fidelity	Complex Word layouts may look different	Set expectations, focus on semantic content, offer "open in Word"
Legacy .doc is not parseable in the browser	Cannot read old files	Treat as P2 with backend conversion, or ask the user to convert
Large PDFs or huge folders	Slowness	Virtualized lists, lazy page rendering, Web Workers
Malicious HTML	XSS	Sanitize, sandboxed iframe, scripts off by default

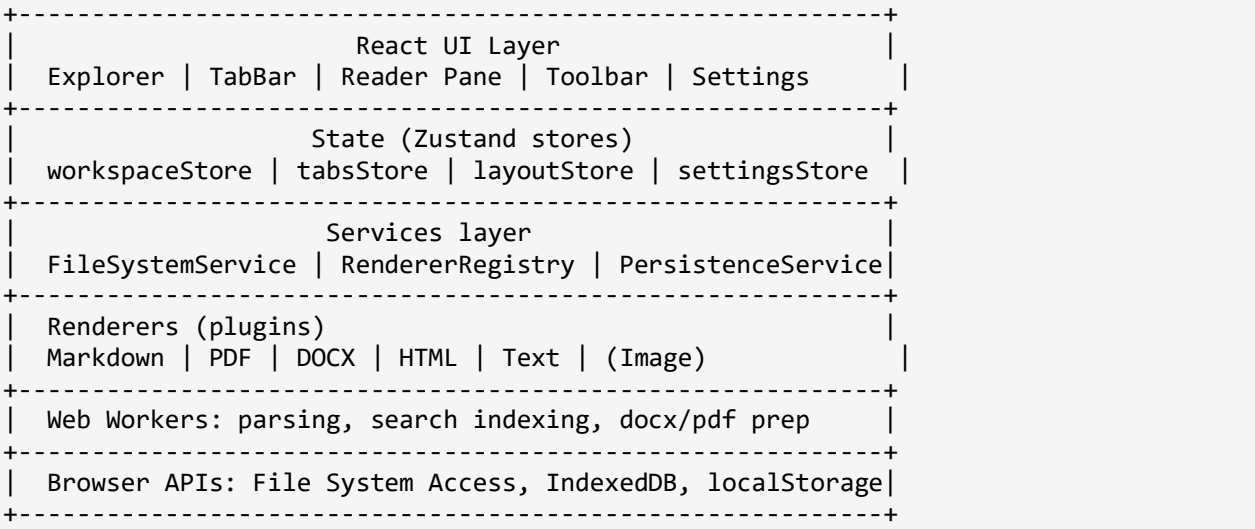
10. Open questions

1. Should v1 be a **web app**, a **desktop app**, or both? (TRD recommends web first, wrap later.)
2. Is editing important, or is this a pure reader? (The sketch shows an Edit toggle, so editing is included for text formats.)
3. Should the app support more formats such as PPTX, EPUB, CSV or images?
4. Single workspace or multiple folders at once?
5. Is this a course/final-year project that needs a specific report format or demo scope?

PART 2 — TECHNICAL REQUIREMENTS DOCUMENT (TRD)

1. Architecture overview

ReadSpace is a **client-side single-page application**. All file reading, parsing and rendering happens in the browser, so no backend is required for the MVP.



A later **desktop shell** (Tauri or Electron) can wrap the same web app to get reliable native file access on every OS. An **optional FastAPI microservice** with headless LibreOffice can add legacy .doc conversion.

2. Recommended technology stack

Concern	Choice	Why
Language	**TypeScript**	Safer, better tooling
Framework	**React 18 + Vite**	You already know React; fast dev and build
State	**Zustand**	Small, simple, no boilerplate
Styling	**Tailwind CSS** + CSS variables for themes	Fast, easy theming
Resizable panels	**react-resizable-panels** (or Allotment)	Production-ready dividers, persistence, collapse
File tree	**react-arborist** or a custom virtualized tree (react-window)	Virtualization for big folders
Tabs drag/drop	**dnd-kit**	Accessible reordering
Markdown	**unified: remark + rehype** (remark-gfm, rehype-highlight,	Safe, extensible, GFM support

Concern	Choice	Why
	rehype-sanitize, rehype-slug)	
Code/text editor	**CodeMirror 6** (lighter) or Monaco (heavier, closest to VS Code)	Edit mode with syntax highlighting
PDF	**pdf.js (pdfjs-dist)**	Standard open-source PDF renderer
DOCX	**mammoth.js** (DOCX to clean HTML)	Reliable semantic conversion
HTML safety	**DOMPurify** + sandboxed `<iframe sandbox srcdoc>`	Prevent XSS
Storage	**IndexedDB via idb** (handles, session), **localStorage** (small settings)	Persist directory handles and workspace
Fuzzy search	**Fuse.js** or **FlexSearch**	Quick open and global search
Testing	**Vitest**, **React Testing Library**, **Playwright**	Unit, component, e2e
Desktop (later)	**Tauri** (small, Rust) or **Electron** (easier JS)	Native FS access, installers
Optional backend	**FastAPI + LibreOffice headless**	Legacy .doc to .docx/.html conversion

3. File system layer

3.1 Strategy

1. **Primary:** the **File System Access API** (`window.showDirectoryPicker()`), available in Chrome and Edge. It returns a `FileSystemDirectoryHandle` that supports listing, reading and **writing back**.
2. **Fallback:** `<input type="file" webkitdirectory>` for Firefox/Safari. This gives read-only access to a flat list of `File` objects with relative paths; editing/saving falls back to "Download as file".
3. **Desktop shell:** native filesystem commands (Tauri `fs` or Node `fs`) with file watching.

3.2 Abstraction

All UI code talks to one interface so the backing implementation can change:

```
interface FileSystemProvider {
  openFolder(): Promise<WorkspaceRoot>;
```

```
  listDir(path: string): Promise<FsEntry[]>;      // lazy, one level
  readFile(path: string): Promise<ArrayBuffer>;
  readText(path: string): Promise<string>;
  writeFile(path: string, data: string | ArrayBuffer): Promise<void>;
  stat(path: string): Promise<{ size: number; modified: number }>;
  watch?(path: string, cb: (e: FsEvent) => void): Unsubscribe;
}

type FsEntry = {
  name: string;
  path: string;            // workspace-relative, '/' separated
  kind: 'file' | 'directory';
  ext?: string;
  size?: number;
};
```

Implementations: `BrowserFsaProvider`, `BrowserFallbackProvider`, `TauriProvider`.

3.3 Persistence of folder access

Directory handles are stored in **IndexedDB**. On restart the app calls `handle.queryPermission()` and, if needed, `requestPermission()` after a user click to restore access without re-picking the folder.

3.4 Tree loading

- Load only the first level on open; load children **on expand**.
- Ignore by default: `node_modules`, `.git`, hidden dot-folders (configurable).
- Virtualize rows so thousands of nodes stay smooth.

4. Renderer plugin system

Each format is a plugin registered by extension and MIME type.

```
interface Renderer {
  id: string;                     // 'markdown' | 'pdf' | 'docx' | 'html' | 'text'
  extensions: string[];           // ['md', 'markdown']
  editable: boolean;              // controls Preview/Edit toggle
  load(file: OpenFile): Promise<RenderModel>;   // parse, may use worker
  Component: React.ComponentType<ViewerProps>;  // the preview UI
  EditorComponent?: React.ComponentType<EditorProps>;
  getOutline?(model: RenderModel): OutlineItem[];
  search?(model: RenderModel, q: string): SearchHit[];
}
```

A `RendererRegistry` maps file extension to renderer; unknown types show a "Cannot preview" screen with an "open as text" option.

4.1 Per-format implementation notes

Format	Pipeline	Key details
Markdown	text → remark (parse) → remark-gfm → rehype → rehype-sanitize → React	Resolve relative image/link paths via the FileSystemProvider (create blob URLs). Syntax highlight code

Format	Pipeline	Key details
		blocks. Generate heading IDs for outline. Parse in a worker for large files.
PDF	ArrayBuffer → pdfjs `getDocument` → render pages to `<canvas>` + text layer	Virtualize pages (render only visible ±1). Run the pdf.js worker. Zoom and fit-width modes. Use text layer for selection and search.
DOCX	ArrayBuffer → `mammoth.convertToHtml` → DOMPurify → render	Map styles to semantic HTML; embed images as data URLs; apply reader CSS. Limited fidelity for complex layouts.
HTML	text → DOMPurify (optional) → `<iframe sandbox srcdoc>`	Default sandbox has no `allow-scripts`; user can enable scripts per file. Base URL handling for relative assets via blob URLs. Toggle "view source".
TXT	text → virtualized line list (or CodeMirror read-only)	Line numbers, wrap toggle, monospace/serif font choice. Handle large files by chunking. Detect encoding (UTF-8 default, fallback via TextDecoder).
.doc (legacy)	Upload to optional FastAPI service → LibreOffice headless → docx/html	Disabled unless the service URL is configured; otherwise show guidance.

5. State management and data models

5.1 Stores

```
// workspaceStore
{
  root: WorkspaceRoot | null;
  tree: Record<string /*path*/, FsEntry[]>;   // lazily filled
  expanded: Set<string>;
  selectedPath: string | null;
  recentFolders: RecentFolder[];
}

// tabsStore
{
```

```
  tabs: Tab[];            // ordered
  activeTabId: string | null;
  closedStack: Tab[];     // for reopen
}
type Tab = {
  id: string;
  path: string;
  title: string;
  rendererId: string;
  mode: 'preview' | 'edit' | 'split';
  pinned: boolean;
  isPreviewTab: boolean;  // italic, replaceable
  dirty: boolean;
  scrollTop: number;
  viewState?: unknown;    // pdf page/zoom, cursor position, etc.
};

// layoutStore
{
  sidebarWidth: number; sidebarVisible: boolean;
  rightPanelVisible: boolean; rightPanelWidth: number;
  zenMode: boolean; preset: 'reading' | 'writing' | 'split';
}

// settingsStore
{
  theme: 'light' | 'dark' | 'sepia' | 'system';
  fontFamily: string; fontSize: number; lineHeight: number;
  contentMaxWidth: number; showHidden: boolean; autoSave: boolean;
}
```

5.2 Persistence mapping

Data	Storage
Directory handles, recent folders	IndexedDB
Open tabs, active tab, scroll positions, per-file view state	IndexedDB (keyed by workspace id)
Layout and settings	localStorage (small JSON)
Unsaved edit drafts	IndexedDB (recoverable after crash)

6. Layout engine

- A top-level CSS grid / flex shell: **TitleBar**, **Workbench**, **StatusBar**.
- The Workbench uses `react-resizable-panels`: `PanelGroup (horizontal)` → `Sidebar Panel` | `Resize Handle` | `Main Panel` | optional `Right Panel`.
- The Main Panel is a column: **TabBar** (fixed height, horizontal scroll) above **ContentArea** (`overflow: auto`, independent scroll). The Preview/Edit toggle is absolutely positioned or sticky at the top-right of ContentArea.

- Panel sizes are saved with `autoSaveId` and mirrored into `layoutStore`.
- Themes use CSS variables (`--bg`, `--fg`, `--accent`, `--sidebar-bg`, `--border`) switched by a `data-theme` attribute.
- Reader typography is controlled by variables (`--reader-font`, `--reader-size`, `--reader-line`, `--reader-width`).

7. Key flows (sequence summaries)

Open folder

1. User clicks Open Folder → `provider.openFolder()` returns a handle.
2. Store handle in IndexedDB; load root level via `listDir('')`.
3. Render tree; restore previous tabs if the folder was seen before.

Open file

1. Click tree node → `tabsStore.openTab(path)`; reuse the tab if it already exists.
2. `RendererRegistry.get(ext)` → `renderer.load()` (worker where possible).
3. Show a skeleton while loading; render `Component` in the content area; restore scroll/view state.

Edit and save

1. Toggle to Edit → mount `EditorComponent` with the file text.
2. Changes update a draft in memory and mark the tab dirty; debounce-save the draft to IndexedDB.
3. Ctrl+S → `provider.writeFile()`; clear dirty flag; refresh the preview.

Restore session

1. On startup read the last workspace id; `queryPermission` on the stored handle.
2. If granted, rebuild tree and tabs; otherwise show a "Reconnect folder" button.

8. Security requirements

- Sanitize all Markdown-derived and DOCX-derived HTML (rehype-sanitize / DOMPurify) with a strict allowlist.
- Render user HTML only inside `iframe sandbox` without `allow-same-origin` and without `allow-scripts` by default.
- Strict **Content-Security-Policy** on the app shell (no inline eval, restricted `connect-src`).
- Treat file names and paths as untrusted: normalize paths, block `..` traversal in any path resolution.
- No network calls with user content in v1; the optional conversion service is opt-in and clearly disclosed.
- Writes only happen to files the user opened, after explicit save.

9. Performance plan

- Lazy tree loading and virtualization for explorer and long text.

- PDF page virtualization; render at device pixel ratio with caching and cancellation of off-screen renders.
- Move heavy parsing (Markdown, DOCX conversion, search indexing) to **Web Workers**; use transferable ArrayBuffers.
- Code-split renderers with `React.lazy`/dynamic import so pdf.js, mammoth and CodeMirror load only when needed.
- Memoize rendered output; cache by `path + modified time`.
- Cap open parsed documents in memory (LRU) and unload inactive tabs' heavy resources.

10. Project structure

```
readspace/
├─ src/
│  ├─ app/                 # App shell, providers, routes
│  ├─ components/
│  │  ├─ explorer/         # FileTree, TreeNode, FilterBox
│  │  ├─ tabs/             # TabBar, Tab, TabOverflowMenu
│  │  ├─ layout/           # Workbench, TitleBar, StatusBar, ResizeHandle
│  │  └─ common/           # Button, Toggle, Dialog, Icon
│  ├─ features/
│  │  ├─ workspace/        # open folder, recent, restore
│  │  ├─ viewer/           # ViewerHost, PreviewEditToggle
│  │  ├─ search/           # quick open, find in doc, global search
│  │  └─ settings/         # theme, reader typography
│  ├─ renderers/
│  │  ├─ markdown/  ├─ pdf/  ├─ docx/  ├─ html/  ├─ text/
│  │  └─ registry.ts
│  ├─ services/
│  │  ├─ fs/               # FileSystemProvider + implementations
│  │  ├─ persistence/      # IndexedDB, localStorage wrappers
│  │  └─ workers/          # markdown.worker.ts, search.worker.ts
│  ├─ stores/              # Zustand stores
│  ├─ styles/              # themes, tokens
│  └─ utils/
├─ tests/                  # unit, component, e2e
├─ public/
└─ vite.config.ts
```

11. Testing strategy

Level	Tools	What to cover
Unit	Vitest	Path utils, renderer registry, store reducers, Markdown/DOCX conversion output
Component	React Testing Library	Tree expand/collapse, tab open/close/reorder, Preview/Edit toggle

Level	Tools	What to cover
E2E	Playwright	Open folder (mocked FS), open mixed files, resize panel, save edit, session restore
Visual	Playwright screenshots	Theme and layout regressions
Manual corpus	50 sample files	Edge-case PDFs, big Markdown, complex DOCX, HTML with scripts
Accessibility	axe-core, keyboard-only pass	Tree, tabs, dialogs

12. Build, deploy and environments

- **Dev:** `vite` dev server; ESLint + Prettier + TypeScript strict; Husky pre-commit.
- **CI:** GitHub Actions running lint, type-check, unit tests and Playwright.
- **Hosting (web):** static hosting on Vercel, Netlify or GitHub Pages (HTTPS is required for the File System Access API).
- **Desktop (later):** Tauri or Electron builds with installers for Windows, macOS and Linux.
- **Optional service:** a Dockerized FastAPI app with LibreOffice for `.doc` conversion.

13. Browser and platform support matrix

Feature	Chrome / Edge	Firefox	Safari	Desktop shell
Open folder with persistent handle	Yes	Fallback (re-pick each time)	Fallback	Yes
Write back to file	Yes	Download only	Download only	Yes
PDF / DOCX / MD / HTML / TXT viewing	Yes	Yes	Yes	Yes
File watching	Polling	No	No	Native

14. Implementation roadmap (suggested)

Week	Milestone
1	Project setup, shell layout with resizable sidebar, theme tokens, FileSystemProvider with folder open and lazy tree
2	Tabs store and TabBar, renderer registry, TXT and Markdown viewers, Preview/Edit toggle with

Week	Milestone
	CodeMirror, save
3	PDF viewer (pdf.js), DOCX viewer (mammoth), sandboxed HTML viewer, session restore
4	Reader settings, outline panel, find in document, quick open, polish, accessibility and tests
5+	Split view, layout presets, global search, desktop wrapper, optional .doc conversion

15. Technical risks

Risk	Mitigation
File System Access API unavailable outside Chromium	Provider abstraction plus read-only fallback and desktop shell
pdf.js bundle size and worker configuration	Dynamic import, copy worker file via Vite config
Relative assets in Markdown/HTML (images, CSS)	Resolve through the provider and create blob URLs; revoke when tab closes
Memory growth with many tabs	LRU cache, unload inactive heavy views
Mammoth drops some Word features	Show a notice, offer download / open externally
Edit conflicts when files change on disk	Compare modified time before save; prompt to reload or overwrite

16. Definition of done for MVP

- Open a folder, expand/collapse the tree, open any supported file into a tab.
- MD, TXT, HTML, PDF and DOCX all render correctly on the 50-file test corpus.
- Resizable sidebar and independent content scrolling work smoothly.
- Preview/Edit toggle works for MD, HTML and TXT, with save to disk.
- Theme switching and session restore work after a browser restart.
- Lint, type-check and the core test suite pass in CI.